SAKUNA.PH

Centralized Historical Disaster Database for the Philippines

SPIDER-ISKOLARS

Members:

Josh Rylie Patiño

Ace Zeus Rimo

Client Name:

Jake Rom Cadag, Ph.D.

(Department of Geography, UP Diliman)

https://joshdotpatinof.github.io/spider-iskolars-blog/

Software Engineering 1

CS 191 WFR1

Instructor: Ligaya Leah Figueroa, Ph.D.

CUSTOMER STATEMENT OF REQUIREMENTS

Project Motivation

The Philippines is one of the most natural hazard-prone countries in the world, frequently facing typhoons, floods, earthquakes, and volcanic eruptions. Over the centuries, a massive amount of disaster data has been accumulated. However, this vital history is highly fragmented. Valuable records are locked in local government offices, paper documents, isolated spreadsheets, or incompatible databases.

To solve this, the **Department of Geography at UP Diliman** established **Sakuna PH**—a centralized, searchable database designed to consolidate historical disaster data. By having a single, unified database of historical hazards, communities, and local planners can make data-driven decisions to mitigate risks and prepare for future calamities.

The Problem & Why We Are Modifying the System

Sakuna PH is already an existing, open-source platform. The database layer is set up, a basic user interface is built, and it is hosted on UP's College of Social Sciences and Philosophy (CSSP) hardware with over 20,000 historical records.

However, the current live system has several operational bottlenecks that limit its usability and growth:

- **Manual Ingestion of Data:** Adding new disaster records is currently an ad-hoc, manual process. There is no structured, self-service form where the public or local officials can easily submit data for admin review.
- **Database Performance Slowdowns:** When users search the historical database, some complex searches (queries) take too long, overloading the CSSP hosting hardware.
- **High Chatbot Operating Costs:** The platform has an AI chatbot helper, but it is unoptimized. Every time a user asks a question, it sends too much background data (words or "tokens") to the AI system, resulting in high monthly billing costs.
- **Isolated Data (Lack of Sharing):** External researchers and disaster warning systems cannot easily query or extract parts of the dataset because there is no public access door (an API) for software-to-software communication.

How the Problem is Solved in Current Practice

Currently, disaster details are scattered across different platforms, each solving only a part of the problem:

- **Project NOAH**: Provides highly visual hazard maps for floods, landslides, and storm surges.
- **GeoRisk Philippines**: Offers local hazard assessments and analytical maps.
- **EM-DAT**: A global emergency database that tracks large-scale macro disaster statistics.
- **Existing Sakuna PH Platform**: Serves as a centralized historical archive going back to the 1600s, but currently lacks smooth user submission tools, optimized speeds, and open sharing capabilities.

Our team, **SPIDER-ISKOLARS**, is extending the existing Sakuna PH codebase to directly address these optimization, data contribution, sharing, and showcase problems

Key Requirements (What the System Will Do)

To solve these issues, the system will deliver the following features:

Legend:

🔴 **High Priority**

🟠 **Medium Priority**

🟡 **Low Priority**

1. The Public Contribution Pipeline (🔴)

A multi-step web form that allows local officials and verified users to submit historical disaster records to the database.

User View

- Accesses the contribution page and fills in the required disaster information.
- Uses input fields with automatic validation for details such as dates, locations, and other record fields.

- Can view the status of their submitted contributions, such as pending, approved, or rejected.

Admin View

- Can view all contributors and their submitted disaster records.
- Can review and verify submitted entries and edit fields when necessary.
- Can approve or reject contributions after review.
- Once an entry is validated, it is automatically logged and added to the disaster database.

2. **Open Data Gateway (REST API & API Guide) (🔴)**

A secure portal allowing researchers and external systems to programmatically request and download disaster records filtered by date, disaster type, or location.

3. **Database Query Optimization (🔴)**

We will reorganize the backend database commands so that searches load much faster and do not freeze the server.

4. **AI Chatbot Cost-Saver (🟠)**

We will optimize the AI chatbot retrieval pipeline. Instead of sending massive, redundant raw texts, the chatbot will use a smarter context retrieval method and cache common questions to avoid redundant paid AI requests.

5. **Interactive Land Page Statistics (🟡)**

High-impact “dashboard cards” on the homepage showing immediate statistics like *Total Disasters*, *Total Casualties* and *Total Financial Damage*. This will be paired with interactive charts showing disaster trends over the years.

6. **Data Science Showcase (🟡)**

A dedicated webpage to feature analysis and data science projects built using Sakuna PH's open data, supporting interactive writeups.

PRELIMINARY UI DESIGN

- **Landing Page Path:** User enters SAKUNA.PH -> Greets user with main statistics cards -> Accesses quick search bar.
- **Contribution Path:** User clicks "Contribute" -> Enters standard information (Disaster Type, Year, Location, Source) -> Preview draft -> Submit to Admin review queue.
- **Data Science Showcase Path:** User clicks "Articles & Showcase" -> Views list of studies -> Opens interactive notebook analysis.

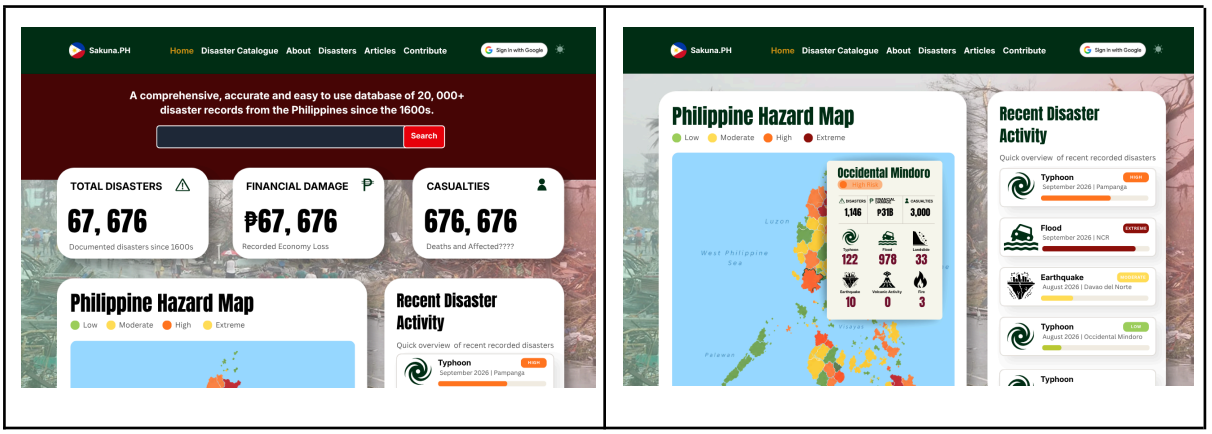


Figure 1: Landing Page UI Mockup

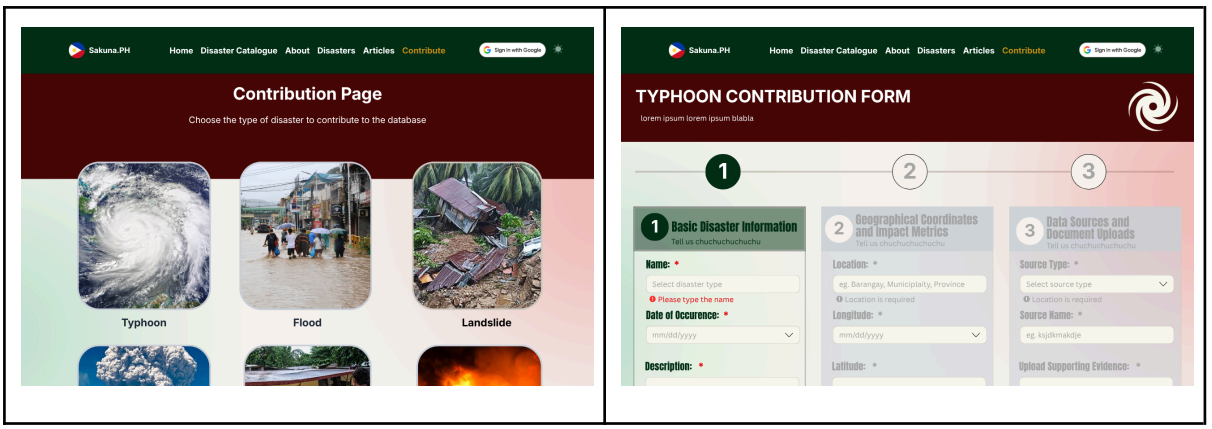


Figure 2: Multi-Step Data Contribution Form

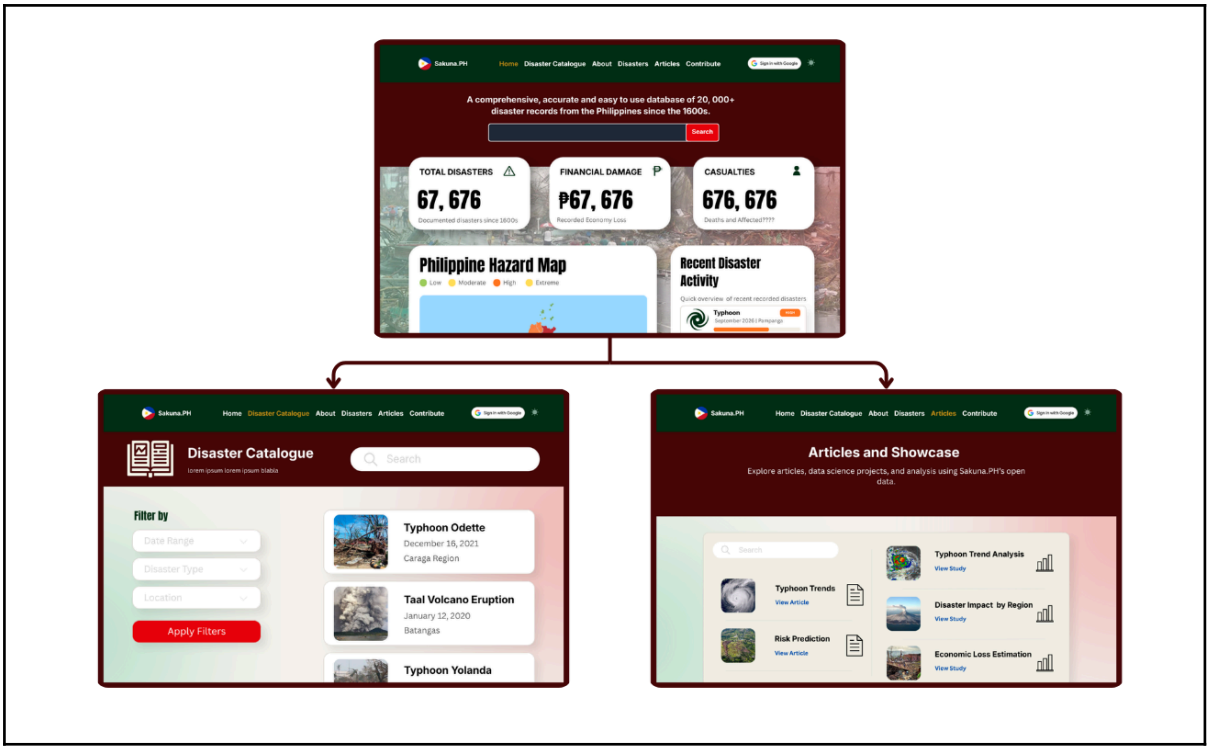


Figure 3: Site Navigation and Site Map Diagram

PLAN OF WORK

A. Estimate Cost and Resources

We estimate the total effort for this refactoring and feature development at **240 person-hours** split equally across our **2 developers** (**120 hours per person**) over the course of the semester.

Why did we adjust this estimate? In real-world software engineering, projects often underestimate the complexity of working with a legacy codebase. To protect our team from delays and technical debt, we built a **25% contingency buffer** directly into our metrics. This buffer covers crucial "Umbrella Activities" like weekly sync meetings with our client, Dr. Cadag, preparing class deliverables, coordinating backend and frontend systems, and deploying the application on college hardware (which often involves unexpected server configuration delays).

To validate our scope, we cross-referenced our total effort using two different software engineering estimation techniques.

Estimation Technique 1: Feature-Based Estimation (Problem-Based)

This technique breaks down the software by modules, sizes them based on estimated Lines of Code (LOC), and applies typical developer productivity speeds for refactoring:

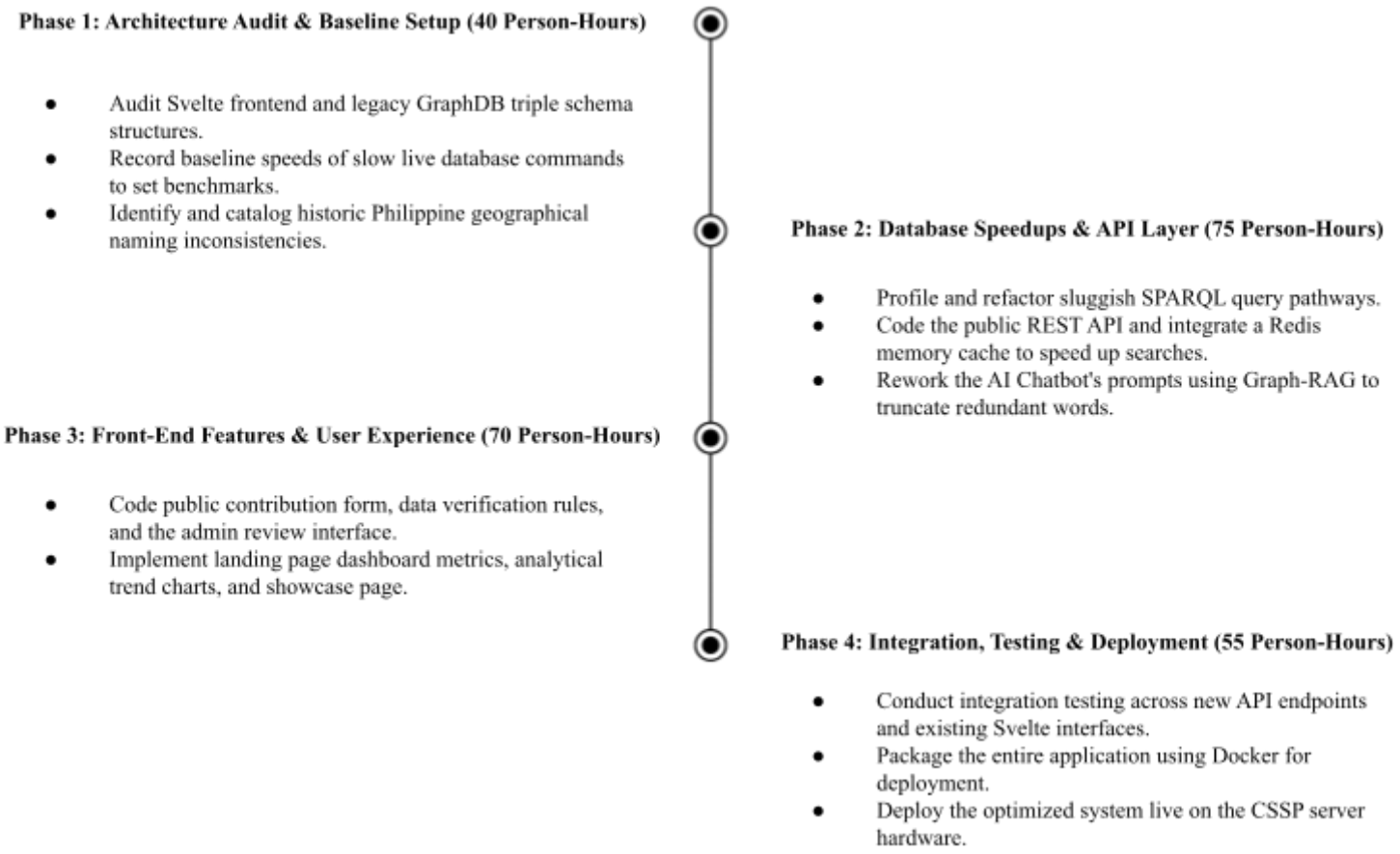
Module / Task Category	Estimated Lines of Code (LOC)	Estimated Person-Hours	What is covered?
Codebase Audit & Speed Optimization Speed-profiling and indexing existing database queries	~500 LOC	45 Hours	Profiling database commands, indexing the GraphDB schema, and benchmarking query speeds.
AI Chatbot Cost-Saver Reworking AI prompt pathways and caching responses	~400 LOC	35 Hours	Restructuring context retrieval using Graph-RAG, trimming down tokens, and implementing local question caching.
Public Contribution Form Form UI, data validation rules, and the admin queue	~900 LOC	55 Hours	Building multi-step forms, inline validation, admin workspaces, and handling historic geographic naming inconsistencies.
Open REST API & Guide	~750 LOC	45 Hours	REST routes, sorting/pagination, and writing the

Building secure data access endpoints and programmer documentation			OpenAPI guide.
Landing Page Dashboard Cards Summary statistics and interactive analytics charts	~600 LOC	35 Hours	Homepage statistics counters and interactive historical trend graphs.
Data Science Showcase Section Interface supporting embedded Jupyter Notebooks & studies	~350 LOC	25 Hours	Interface supporting markdown studies, embedded notebooks, and user guides.
Total Project Scope	**~3,500 LOC**	**190 Person-Hours**	Includes full legacy integration, documentation, and client review overhead.

Note: Productivity rates range from 10 LOC/hour for highly conceptual database profiling to 16 LOC/hour for standard visual Svelte components, which includes research, local coding, manual testing, and debugging.

Estimation Technique 2: Process-Based Estimation (Work Phases)

This technique tracks effort through sequential software engineering phases, allocating realistic times for planning, execution, and deployment:



Both estimation routes independently align at 240 person-hours, giving our team high confidence that this plan is realistic, bulletproof, and safe from underestimation.

System Resource Requirements

To run this application seamlessly, the target hardware environment must meet the following constraints:

- **Hosting Server:** College of Social Sciences and Philosophy (CSSP) Linux server infrastructure.
- **Database Engine:** Ontotext GraphDB managing over 20,000 RDF triples.
- **Server Runtime:** Node.js (for SvelteKit routes) and a Redis in-memory cache layer.
- **User Hardware:** A standard monitor or mobile screen supporting at least **1024 × 768 pixels** with a modern web browser.
- **Storage Constraint:** At least **20 GB** of dedicated disk space.

- **Network Speed Constraint:** A minimum network bandwidth of **10 Mbps**.

B. Schedules and Milestones

The team will execute the project in bi-weekly increments. Below is the milestone schedule:

Spider-Iskolars_GanttChart

C. Project Metrics (How We Measure Success)

To track our progress and prevent delays, we will simplify our metrics to **two primary measures:**

1. **Sprint Velocity (Team Delivery Rate):**

The percentage of planned software tasks that we finish on time during our bi-weekly development cycles.

Goal : We aim to complete at least **90%** of our committed tasks in every sprint to ensure no milestone dates slip.

2. **System Search Speed (Database Speedup Rate):**

The percentage of time saved when querying historical disaster records compared to the original, unoptimized setup.

Goal : A minimum of **40% speedup** (reduction in database load and latency) measured on the CSSP server hardware.

D. Effort Breakdown (Who Does What)

Josh Rylie Patiño (Developer / Co-Lead)

- Responsible for checking existing database code, identifying search speed bottlenecks, and indexing the database.

- Responsible for building the REST API, caching system, and restructuring the AI chatbot logic using Graph-RAG.
- Responsible for checking existing Svelte/SvelteKit user interfaces.

Ace Zeus Rimo (Developer / Co-Lead)

- Also responsible for checking existing database code, identifying search speed bottlenecks, and indexing the database.
- Manages the final live deployment of the platform onto CSSP servers
- Responsible for designing and coding the public contribution form, admin verification workspace, landing page stats cards, charts, and the data science showcase view.

Joint Responsibilities: Integrating frontend and backend systems, testing endpoints, user manuals and developer documentation, and client coordination.

PROJECT RISKS

We have simplified our risks into a clear, understandable matrix focusing on the core issues:

Risk	Likelihood	Impact on Project	How We Plan to Avoid or Fix It (Mitigation)
Slow Database Searches on CSSP Hardware	Medium	High	Profile slow database commands early. Add a caching layer (Redis) to instantly load common numbers, and set query limits.
High Billing Costs	High	High	Use Graph-RAG context

for the AI Chatbot			truncation to send fewer background words, cache common query answers locally, and put strict limits on how many questions a user can ask per session.
Spam or Fake Disaster Submissions	Medium	High	Implement strong field checks in the form (e.g. valid dates), force all user submissions to wait in a queue, and require manual admin approval before any data goes live.
Integration Issues Breaking Existing Website	Medium	Medium	Run a thorough codebase audit in Milestone 1. Write modular backend pathways that do not interfere with existing stable code.
Server Downtime / Lack of Server Access	Low	Medium	Package the entire application inside Docker containers. This lets us develop, test, and preview everything on our local laptops exactly as it would run on the actual UP server.

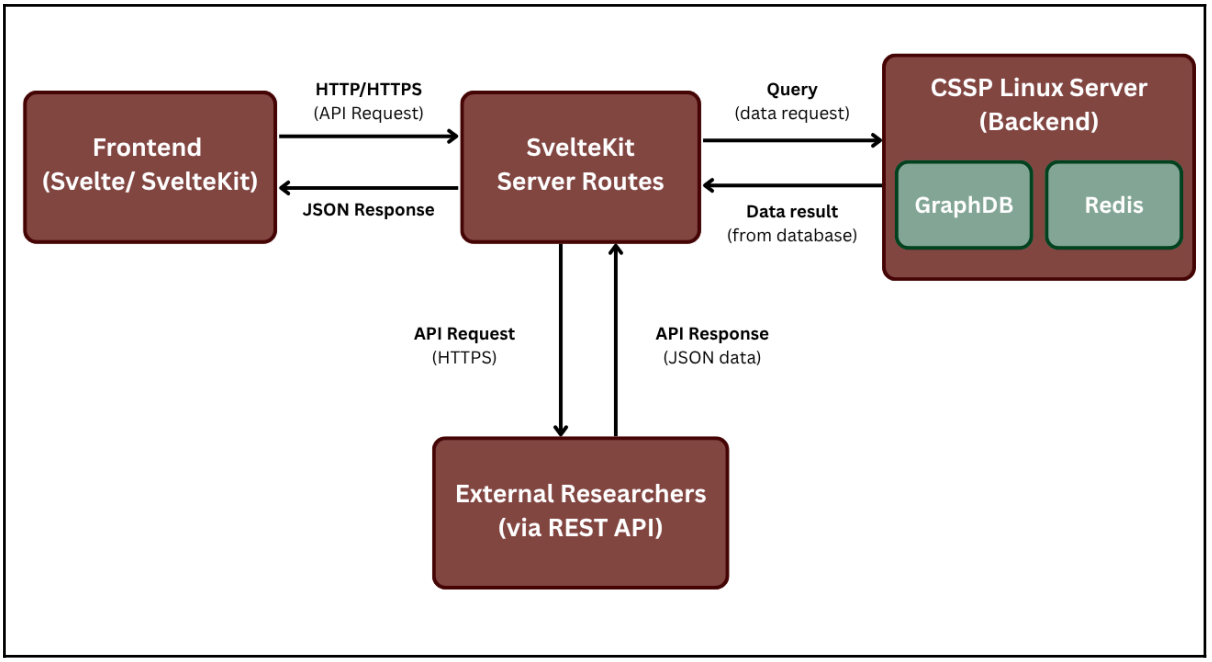


Figure 4: System Architecture Diagram